Resilience Alpha

DOSSIER D’INGÉNIERIE · ARCHITECTURE · EXPLOITATION

Prototype local de paiement fictif et de cyber-résilience Documentation technique · Version 1.0 · 23 octobre 2026

Périmètre. Démonstrateur Docker Compose, authentification faciale locale sur le PC Hôte, signalisation ESP8266, détection AIOps et procédures de reprise.

Public. Équipe projet, exploitants du laboratoire, évaluateurs et lecteurs techniques.

Avertissement. Les transactions et scénarios sont synthétiques. Ce document décrit un prototype pédagogique ; il ne constitue ni une homologation bancaire, ni une certification biométrique, ni une déclaration de conformité réglementaire.

Lecture du dossier

Ce dossier suit le parcours du système : objectifs et périmètre, architecture, interfaces et sécurité, fonctions d’authentification, surveillance, reprise, validation et préparation du déploiement. Les résultats mesurés sont distingués des objectifs et des essais restant à réaliser.

À retenir : le système traite des paiements fictifs ; les services métier restent sur un réseau Compose interne ; seules les interfaces du dashboard et du WAF sont publiées par défaut.

1. Synthèse et périmètre

Objectif

Resilience Alpha met en scène un service de paiement fictif auquel sont associés une interface opérateur, une authentification renforcée, une supervision et une reprise automatisée d'un incident applicatif contrôlé. Le projet permet d'observer une chaîne complète : activité nominale, dégradation, collecte de preuves, décision de détection, action limitée, puis vérification métier.

La consigne additionnelle renforce la connexion du back-office : l'utilisateur fournit son identifiant, est vérifié par la caméra USB du PC Hôte, puis saisit son mot de passe. L'administration permet de gérer les comptes et profils faciaux. Un ESP8266 équipé d'une LED rouge/verte fournit un retour physique des étapes de connexion.

Périmètre fonctionnel

- Interface React servie par Nginx pour le statut, les transactions, les événements et l'administration des habilitations.
- Authentification FastAPI avec rôles `admin` et `operator`, défi facial à usage unique et jeton opérateur à durée limitée.
- API de paiement FastAPI : validation, persistance, santé, métriques et routes d'exercice/remédiation séparées.
- PostgreSQL pour les paiements ; SQLite pour les comptes et l'état local de verrouillage facial.
- Prometheus, Loki et Grafana pour les métriques, événements et preuves.
- Isolation Forest au sein d'un détecteur AIOps qui n'a ni socket Docker ni accès direct à la base.
- Agent webcam sur le PC Hôte et firmware PlatformIO sur ESP8266, relié par USB série.
- Profil optionnel Ollama et assistant RAG documentaire, séparé du chemin de remédiation.

Hors périmètre

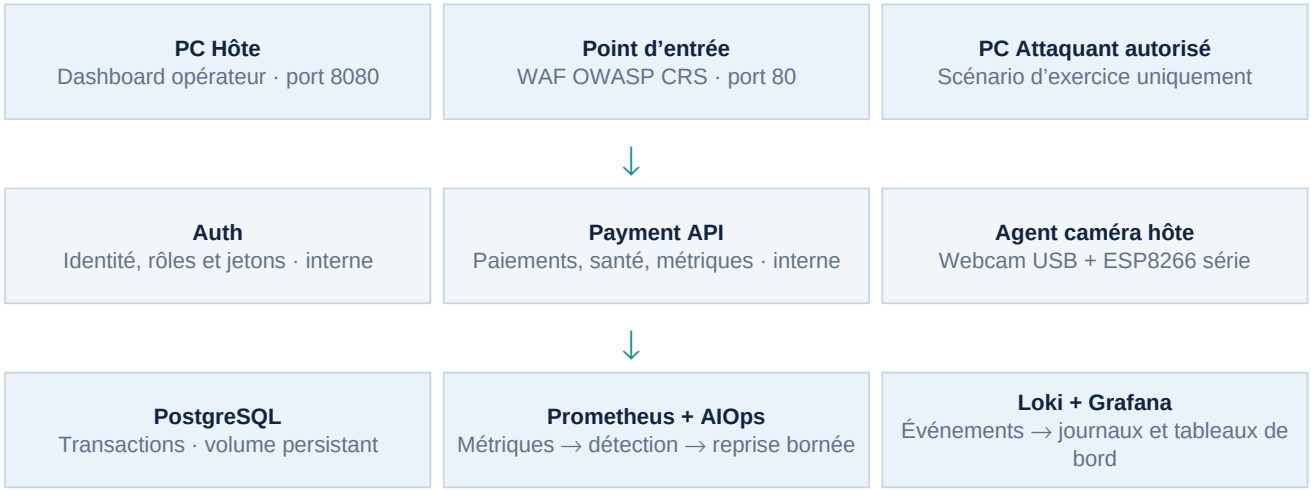
Aucun paiement, compte bancaire ou jeu de données de production n'est utilisé. Le prototype ne remplace pas un fournisseur d'identité, une plateforme de paiement, une supervision d'entreprise, un dispositif biométrique certifié ou un plan de continuité approuvé. Le scénario n'évalue pas une panne physique complète, une résistance générale aux attaques ni un réseau non isolé.

Principes de conception

Minimiser l'exposition. Auth, API, base, Loki et AIOps ne publient pas de port hôte. **Séparer les capacités.** Opérateur, exercice et remédiation emploient des identités distinctes. **Vérifier la reprise.** Une décision ML n'est pas une preuve de restauration : santé et transaction de contrôle sont requises. **Rendre les limites visibles.** Les résultats synthétiques et les essais localhost ne sont pas présentés comme une recette sur deux machines.

2. Architecture générale

Vue logique



Les services Docker communiquent sur le réseau Compose interne. Seuls le dashboard et le WAF sont publiés par défaut ; l’agent caméra reste sur le PC Hôte.

Rôles et déploiement

Le dashboard et le WAF forment les interfaces d’entrée ; Auth porte l’identité et les défis ; la Payment API applique les règles métier et persiste dans PostgreSQL. Prometheus et Loki collectent respectivement les métriques et événements. AIOps lit les métriques et ne peut demander qu’une reprise applicative bornée. Grafana présente les sources Prometheus et Loki. L’ESP8266 affiche les états du parcours facial.

Le déploiement cœur est décrit dans `docker-compose.yml` et utilise le réseau bridge interne `resilience-net`. Le profil optionnel `ai` ajoute Ollama, Chroma et l’assistant RAG ; il n’est pas requis pour les paiements ou la reprise.

Deux zones d’exécution

PC Hôte. Docker Compose, webcam USB, processus `face_agent` et carte ESP8266. Le service d’authentification en conteneur rejoint l’agent via `host.docker.internal:8765` et un jeton dédié.

PC Attaquant autorisé. Émet uniquement le trafic d’exercice documenté, sur le réseau de laboratoire du groupe. Il utilise `EXERCISE_TOKEN`, jamais le secret de remédiation. Un test sur localhost ne prouve pas le fonctionnement entre deux machines.

Dépendances structurantes

Le WAF dépend de l’API, l’API d’Auth et de PostgreSQL ; AIOps interroge Prometheus, puis les routes internes d’Auth et de l’API. Une panne de télémétrie ne suffit pas à conclure à une panne métier. Ansible valide santé et scrape, sans remplacer le test fonctionnel.

3. Interfaces, réseau et frontières de confiance

Ports et exposition

Interface	Publication par défaut	Usage
Dashboard	`127.0.0.1:8080`	Accès opérateur local
WAF	`127.0.0.1:80`	Entrée HTTP vers l'API
Prometheus	`127.0.0.1:9090`	Consultation locale des métriques
Grafana	`127.0.0.1:3000`	Tableaux de bord locaux
Auth, API, PostgreSQL, Loki, AIOps	Aucune publication hôte	Communications réseau Compose
Agent caméra	TCP 8765 du PC Hôte	Appels authentifiés depuis Auth et signal LED

Pour une démonstration depuis un second PC, `WAF_BIND_ADDRESS` et `DASHBOARD_BIND_ADDRESS` peuvent être liés à l'interface du point d'accès dédié. Cette ouverture est une exception de laboratoire : configurer le pare-feu pour le seul réseau privé, puis vérifier `docker compose ps`. Ne pas exposer les ports de gestion ou le service caméra à un réseau tiers.

Identités et capacités

Identité	Autorisations prévues	Limites principales
Opérateur	Paiement, lecture d'état métier	Aucun contrôle d'incident ou de remédiation
Administrateur	Gestion des comptes et profils faciaux	Usage limité au back-office
PC d'exercice	Injection d'une panne de laboratoire	Pas de remédiation ni de contrôle système
AIOps	Détection et remise à zéro de la panne contrôlée	Pas d'injection, de commande Docker ou d'accès PostgreSQL
Agent caméra	Vérification faciale et pilotage LED	Accès au matériel local, protégé par `FACE_AGENT_TOKEN`

Les secrets `EXERCISE_TOKEN`, `REMEDIATION_TOKEN`, `FACE_AGENT_TOKEN`, `AIOPS_CLIENT_SECRET` et la clé de signature doivent être distincts et fournis depuis une configuration locale non versionnée. La clé de chiffrement des profils faciaux doit également rester hors du dépôt.

Limites cryptographiques et réseau

Les échanges du démonstrateur sont en HTTP sur un réseau de laboratoire isolé ; jetons et identifiants ne doivent pas circuler sur un réseau non fiable. Les secrets Compose sont transmis par variables d'environnement. Les jetons HMAC et secrets partagés facilitent la démonstration mais ne forment pas un système d'identité ou de gestion de secrets de production. Un déploiement réel nécessiterait notamment TLS, rotation et stockage dédiés.

4. Parcours de paiement et données

Transaction nominale

1. L'opérateur s'authentifie et obtient un jeton Bearer d'une heure. Le navigateur le conserve dans `sessionStorage`.
2. Le dashboard soumet une demande de paiement au WAF ; l'API valide l'identité, le montant et les champs métier.
3. En fonctionnement nominal, l'API enregistre la transaction dans PostgreSQL et retourne son identifiant au dashboard.
4. L'API mesure la durée et le statut HTTP, expose `/metrics` et publie les événements applicatifs dans Loki.
5. Le dashboard rafraîchit la santé et, pour une session authentifiée, l'état opérationnel agrégé.

Données persistées

Donnée	Stockage	Remarque
Paiements simulés	PostgreSQL, volume `postgres-data`	Identifiant, utilisateur, montant, statut
Comptes, rôles, empreintes de mot de passe	SQLite du service Auth	Mot de passe dérivé par PBKDF2-HMAC-SHA256
Profils d'empreintes faciales chiffrés	SQLite local de l'agent	Chiffrement Fernet avec `FACE_DATA_KEY`
Métriques et événements	Volumes Prometheus et Loki	Rétention configurée limitée pour le laboratoire
Index documentaire optionnel	Volume Chroma	Documents Markdown montés en lecture seule

Les photos d'enrôlement sont traitées localement ; les empreintes extraites sont chiffrées et les images ne sont pas conservées dans la base. Aucune donnée biométrique n'est envoyée au navigateur ou à l'API de paiement. Le dossier local de photos de référence ne doit pas être intégré à l'archive de livraison.

Injection d'incident et intégrité métier

La panne d'exercice est une condition applicative contrôlée, limitée dans le temps. Lorsque l'incident est actif, l'API renvoie HTTP 503 aux appels concernés avant toute écriture d'une transaction échouée. La route d'injection exige le jeton d'exercice ; elle n'est pas une route d'exploitation arbitraire.

La reprise retire cette condition par une route interne limitée, puis AIOps vérifie la santé et réalise une transaction synthétique de contrôle de 0,01 EUR. Une transaction de contrôle confirmée et un état de santé nominal sont nécessaires avant de déclarer la reprise.

Conservation et sauvegarde

PostgreSQL utilise un volume persistant, mais ce fait ne constitue pas une sauvegarde. Une stratégie de sauvegarde, de restauration PITR, de rétention et de vérification de cohérence reste à définir et tester avant toute utilisation réelle. Les résultats de démonstration doivent être rapprochés des horodatages, identifiants de transaction, journaux et métriques.

5. Authentification faciale et ESP8266

Séquence d’accès au back-office

1. L’utilisateur saisit son identifiant. Auth demande à l’agent du PC Hôte d’ouvrir la webcam USB.
2. YuNet détecte le visage et SFace calcule une empreinte locale comparée au profil autorisé ; plusieurs captures sont évaluées.
3. En cas de correspondance, l’agent libère la caméra, un défi à usage unique est créé pour deux minutes et le mot de passe est demandé.
4. Un mot de passe valide délivre un jeton opérateur d’une heure. Un échec efface le défi et ramène à l’identifiant.
5. Deux échecs faciaux consécutifs verrouillent l’identifiant pendant cinq minutes. Le compte et le verrouillage sont suivis dans SQLite.

États LED

Événement	LED	Durée / transition
Attente ou annulation	Éteinte	État de repos
Visage reconnu, mot de passe attendu	Verte clignotante	Jusqu’au résultat ou expiration du défi
Connexion acceptée	Verte fixe	30 secondes, puis extinction
Visage non reconnu	Rouge fixe	30 secondes, puis extinction
Mot de passe incorrect	Dix impulsions rouges	Puis extinction et retour à l’identifiant

Matériel et liaison

La carte NodeMCU ESP8266 pilote la LED verte sur D1 et la rouge sur D2, chacune avec une résistance série de 220 Ω. L’agent conserve le port USB série ouvert à 115200 bauds et transmet des états textuels au firmware. Arrêter l’agent avant un téléversement PlatformIO ou l’ouverture d’un moniteur série. Le firmware actuel communique par USB série ; il n’utilise pas de Wi-Fi ou de broker MQTT.

Écart à la consigne additionnelle : celle-ci demande que l’ESP8266 rejoigne le point d’accès en Wi-Fi. Cette liaison n’est pas implémentée dans la version actuelle, qui utilise USB série ; l’exigence Wi-Fi reste donc à réaliser ou à faire valider explicitement.

Habilitations et limites biométriques

Un administrateur peut créer un compte `admin` ou `operator`, enrôler de nouveau son visage ou supprimer un compte et son profil depuis le dashboard. L’enrôlement doit être supervisé et réalisé avec le consentement de la personne. La perte de `FACE_DATA_KEY` rend les profils existants illisibles.

YuNet/SFace est une reconnaissance locale de démonstration, sans détection de vivacité. Le seuil de similarité par défaut (0,363) n’est pas une mesure de sécurité ou de performance. Il faut mesurer faux rejets et fausses acceptations dans des conditions contrôlées ; ne pas employer ce prototype seul pour protéger un système réel.

6. Observabilité et détection AIOps

Chaîne de mesure

Prometheus scrape l’API toutes les cinq secondes. Les séries comprennent le nombre de requêtes de paiement par statut, la durée, l’état d’incident, la dernière décision d’anomalie et la vérification de reprise. Les événements métier et erreurs sont envoyés vers Loki. Grafana provisionne des panneaux de débit, taux de 5xx, latence, chronologie et états de reprise. Les journaux AIOps et le dashboard donnent un contexte opérateur complémentaire.

Détection

Le détecteur interroge des fenêtres Prometheus de 60 secondes. Le vecteur utilisé, dans l’ordre, comprend :

- nombre de demandes de paiement ;
- ratio des réponses HTTP 5xx ;
- durée moyenne de traitement, en secondes.

Un Isolation Forest entraîné au démarrage sur 800 observations synthétiques nominales détermine si l’observation est atypique. La collecte a lieu toutes les trois secondes par défaut. L’AIOps exige aussi au moins trois requêtes et un ratio 5xx d’au moins 25 % avant d’agir ; une prédiction seule ne suffit pas.

Décision et action

La décision, son score et ses caractéristiques sont consignés via l’API métier. AIOps demande un jeton de service à Auth et utilise son secret de remédiation distinct pour retirer uniquement la panne contrôlée. Le composant ne peut pas redémarrer arbitrairement les conteneurs : il n’a pas de socket Docker, de privilèges Linux élevés ou d’accès direct à PostgreSQL.

Après l’action, AIOps vérifie `/health`, crée une transaction métier de contrôle et rapporte l’issue. Une échec de collecte Prometheus n’entraîne pas une reprise automatique. Les nouvelles tentatives sont espacées ; l’état et le résultat restent visibles dans les événements.

Interprétation des preuves

Les métriques expliquent le comportement agrégé ; Loki aide à reconstituer la chronologie ; PostgreSQL établit les transactions persistées. Ils doivent être rapprochés par horodatage et identifiant. Une alerte ou une prédiction ML ne prouve ni la cause racine, ni l’intégrité d’une sauvegarde, ni une restauration générale.

7. Réponse à incident et continuité

Procédure opérateur

Avant. Vérifier le réseau de laboratoire autorisé, le `.env` local, les services sains, le dashboard, Grafana et les journaux. Réaliser une transaction fictive nominale et ouvrir la supervision.

Pendant. Depuis le seul PC Attaquant autorisé, lancer le script d'exercice avec `EXERCISE_TOKEN`. Observer les réponses 503, le ratio d'échecs, les événements d'incident et la prédiction. Ne pas transmettre le secret de remédiation au PC Attaquant.

Reprise. Vérifier le journal AIOps et l'action limitée ; confirmer le retour de `/health` et la transaction de contrôle. Le scénario doit aussi réussir une nouvelle transaction d'exercice. Si la reprise échoue ou dépasse le délai de 90 secondes prévu par le script, interrompre la démonstration, conserver les preuves et appliquer la procédure supervisée ; ne pas présenter une intervention manuelle comme une action autonome.

Critères de recette du scénario

- Huit paiements d'exercice sont émis et au moins un reçoit 503 pendant la panne.
- Les métriques et le journal montrent les caractéristiques utilisées et la décision.
- La remédiation suit la détection sans commande corrective humaine.
- La santé revient à `ok`, la transaction de contrôle est acceptée et une nouvelle transaction d'exercice réussit.
- Les événements, horodatages, identifiants et montants concordent entre API, dashboard, Grafana/Loki et base.

Cibles et limites PCA/PRA

Le script impose un objectif de retour nominal inférieur à 90 secondes ; cette valeur est une cible de test, pas une garantie. La panne contrôlée rejette avant écriture, ce qui donne une cible de perte nulle pour cet exercice précis, pas un RPO garanti en cas de perte du volume. La sauvegarde et la restauration PostgreSQL ne sont pas encore qualifiées.

Le repli consiste à arrêter l'exercice, préserver les journaux, documenter les heures et utiliser une reprise supervisée. Les rôles doivent être explicites : l'opérateur consigne, AIOps agit dans son périmètre et l'assistant documentaire conseille sans administrer.

Préservation des preuves

Conserver les identifiants de transactions, l'heure d'injection, les séries de métriques, la décision, l'action et le résultat de vérification. Distinguer les logs de test des résultats réels. La capture demandée par le sujet doit être enregistrée en continu depuis le dispositif autorisé ; elle ne peut être remplacée par une reconstruction ou par un test localhost.

8. Analyse de risques et sécurité opérationnelle

Registre préliminaire

Risque	Mesures présentes	Risque résiduel / preuve à obtenir
Indisponibilité du paiement	Injection bornée, métriques, reprise vérifiée	Répéter le test réseau et mesurer plusieurs cycles
Secret compromis	`.env` local, rôles et jetons séparés	Rotation manuelle ; pas de coffre runtime ni rotation automatique
Perte ou altération de base	Volume PostgreSQL persistant	Sauvegarde, restauration et cohérence non qualifiées
Détection erronée	Isolation Forest et seuils métier	Données synthétiques ; mesurer sur un jeu autorisé représentatif
Exposition réseau	Ports de gestion liés à localhost	Contrôler pare-feu, point d'accès et ports avant chaque exercice
Atteinte à la confidentialité faciale	Traitement local, empreintes chiffrées	Pas de vivacité ; consentement, conservation et seuils à valider
Privilège excessif AIOps	Pas de socket Docker, endpoint borné	Secrets partagés et confiance de service à renforcer

Mesures d'exploitation

- Ne jamais committer `.env`, clé privée, `FACE_DATA_KEY` ou profils biométriques.
- Renouveler tout secret qui aurait été partagé ; retirer un fichier du dernier commit ne purge pas l'historique.
- Garder le réseau de démo isolé ; confirmer les ports réellement publiés avec Compose.
- Ne pas activer l'injection hors laboratoire ; garder `EXERCISE_TOKEN` et `REMEDIATION_TOKEN` distincts.
- Vérifier les volumes, journaux et alertes ; une rétention technique limitée n'est pas une politique d'archivage.
- Examiner les erreurs de reconnaissance faciale et disposer d'une procédure de nouvel enrôlement.

Cadre réglementaire

Le document de risques du dépôt est une pré-analyse, pas un atelier EBIOS RM complet. Propriétaires, sources de risque, niveaux, acceptation et plan de traitement doivent être validés par le consortium. Les sujets NIS2 et DORA sont des pistes d'analyse : l'applicabilité dépend de l'entité et de son activité. Le prototype et un exercice local ne démontrent pas la conformité à ces réglementations.

Traitement recommandé avant industrialisation

Établir le BIA, faire approuver RTO/RPO, éprouver les sauvegardes et restaurations, adopter TLS et un gestionnaire de secrets, assurer la rotation et l'identité des services, renforcer les contrôles d'accès, mesurer les performances biométriques avec consentement et définir rétention, audit et notification. Ce travail dépasse le périmètre pédagogique.

9. Modèle ML et validation

Données et paramètres

Le modèle `sklearn.ensemble.IsolationForest` est réentraîné à chaque démarrage sur 800 observations nominales synthétiques, avec `random_state=42` et `contamination=0.01`. Les caractéristiques sont volume Poisson de paramètre 3, ratio d'erreurs Beta(0,6 ; 35) et latence normale de moyenne 0,1 s et écart-type 0,025 s, bornée entre 0,025 et 0,25 s. Pour une fenêtre vide, ratio et latence sont nuls.

Le jeu d'évaluation déterministe distinct utilise 400 fenêtres nominales et 200 fenêtres incidentes, graine 2026. Les incidents combinent volume Poisson de paramètre 10 et ratio Beta(10 ; 2) ; leur latence reste dans la plage nominale. Le déclenchement exige en plus au moins trois requêtes et un ratio 5xx de 25 %.

Résultat disponible

Le test synthétique documenté observe 16 faux positifs sur 400 fenêtres nominales (4 %) et 0 faux négatif sur 200 incidents (0 %), dans le jeu et avec les paramètres décrits. Le seuil de test est de 5 % pour chaque taux. C'est un résultat reproductible sur données artificielles, pas une estimation de performance en trafic réel.

Plan de vérification

1. Exécuter les tests unitaires API et détecteur avec les dépendances consignées.
2. Valider la configuration Compose sans afficher les secrets.
3. Tester le parcours nominal, le refus sans jeton et les rôles de chaque capacité.
4. Réaliser l'exercice contrôlé ; recueillir mesures, journaux, décision et résultat.
5. Tester les échecs de collecte, WAF, caméra, mot de passe, ports et restauration séparément.
6. Répéter sur deux machines au point d'accès du laboratoire avant de revendiquer cette preuve.

Résultats mesurés, sans extrapolation

Un essai local consigné le 20 octobre 2026 a réalisé une transaction nominale, huit réponses 503 pendant l'injection, puis une remédiation et une transaction de contrôle réussie. Le délai injection → vérification était d'environ **6,1 secondes**. Il s'agit d'un seul essai localhost, pas d'un RTO garanti, d'une preuve réseau à deux machines ou d'un résultat biométrique.

Les tests automatisés ne valident pas les images Docker, le WAF, le réseau Windows, le matériel réel, une base restaurée ou une durée de reprise générale. Toute nouvelle mesure doit inclure le protocole, la configuration, l'environnement, les échecs et le nombre de répétitions.

10. Déploiement et exploitation

Prérequis

PC Hôte avec Docker Desktop et Compose v2, Python 3.12 pour l'agent et les tests, webcam USB et ESP8266 connecté. La démo à deux postes nécessite le point d'accès isolé et un PC Attaquant autorisé. Prévoir les ressources disque et mémoire avant d'activer le profil Ollama/RAG.

Mise en service cœur

1. Créer `.env` à partir de `.env.example`, générer des secrets distincts et ne pas les afficher dans les journaux.
2. Valider `docker compose --env-file .env config --quiet`.
3. Démarrer avec `docker compose --env-file .env up --build -d` et vérifier `docker compose ps`.
4. Confirmer la santé des services, les ports publiés et la cible Prometheus.
5. Démarrer séparément l'agent caméra sur le PC Hôte, télécharger/vérifier les modèles et téléverser le firmware PlatformIO.
6. Enrôler localement l'administrateur avec consentement ; vérifier le parcours webcam, mot de passe et LED.
7. Ouvrir dashboard, Grafana et journaux avant d'exécuter le scénario.

Validation et administration

Les tests API/AIOps utilisent une base temporaire et se lancent par `pytest api/tests aiops/test_detector.py -q`. Le dashboard se valide par `npm ci` puis `npm run build`. Ansible déploie la configuration existante et vérifie des contrôles de santé ; ce playbook n'effectue pas de sauvegarde ou de restauration PostgreSQL.

Pour le profil documentaire, `docker compose --profile ai ...` active Ollama et l'assistant RAG. Le service indexe des documents montés en lecture seule ; il ne possède pas de route de remédiation. Vérifier ses sources et la disponibilité des modèles avant l'exercice.

Arrêt et opérations matérielles

Arrêter l'agent avant de reprogrammer l'ESP8266 ou de libérer la webcam. Ne pas lancer un moniteur série concurrent. Pour un incident réel ou un échec de reprise pendant la démonstration, interrompre le trafic d'exercice, préserver les preuves et basculer sur la procédure supervisée ; ne pas masquer une correction manuelle.

11. Traçabilité et conclusion

Couverture des attentes du sujet

Attente	Éléments présents	Preuve ou action restante
Paiement, API, authentification et base	Dashboard, Auth, Payment API, PostgreSQL, tests de cycle	Démontrer une transaction depuis l'interface
Exercice contrôlé depuis un second poste	Script dédié et jeton séparé	Réaliser et filmer l'essai sur le point d'accès
Détection ML depuis les métriques	Prometheus, Isolation Forest, seuils explicites	Rejouer et archiver les résultats sur l'environnement final
Remédiation et vérification métier	Endpoint borné, santé, transaction de contrôle	Capturer la séquence réelle et ses horodatages
Supervision	Dashboard, Prometheus, Loki, Grafana	Vérifier les panneaux déployés
Caméra USB hôte et gestion des profils	Agent facial local, rôles, enrôlement et verrouillage	Évaluer les faux rejets/fausses acceptations avec consentement
ESP8266, LED bicolore et Wi-Fi demandé	Firmware PlatformIO, états reçus par USB série	Le Wi-Fi au point d'accès n'est pas implémenté ; vérifier câblage et temporisations sur matériel réel
Déploiement, RAG, risques et continuité	Compose, Ansible, profil IA, documents de gouvernance	Valider les risques, la restauration et l'applicabilité réglementaire

Livrables de remise

Le sujet prévoit un dossier d'ingénierie, une présentation, un clip continu d'au plus 60 secondes, une archive de code sans secrets et un démonstrateur sur deux machines. Le présent PDF couvre le dossier technique et son architecture. Le numéro d'équipe, les noms et contributions des membres, les captures réelles et le clip ne sont pas disponibles dans le dépôt et ne sont pas inventés ici.

Conclusion

L'architecture sépare les interfaces publiées des services métier internes, différencie les capacités d'opérateur, d'exercice et de remédiation, et associe la détection à une vérification métier. L'extension faciale et matérielle reste locale au PC Hôte. Les résultats actuels prouvent un scénario logiciel local et synthétique ; la démonstration sur deux postes, l'évaluation biométrique et la restauration restent des validations à réaliser.

Références internes : `docs/architecture.md`, `docs/face-auth-and-iot.md`, `docs/incident-response.md`, `docs/ml-experiment.md`, `docs/risk-pca-pra.md`, `docs/security-operations.md`, `docs/testing-and-demo.md`, `docs/requirements-traceability.md`, `rag/procedure_crise.md`.